# Actuator-Aware Geometric Flight Control of a Quadrotor

MCAV project — Technical report — October 2026

Victor Gabriele De Matteis

This project delivers a flight controller for a 0.8 kg quadrotor that takes off and tracks a circular trajectory in the MuJoCo physics engine. The controller works directly on rotation matrices (no Euler-angle singularities). Its gains are not hand-tuned: they are computed by convex optimisation (linear matrix inequalities) so that every motor stays inside its 0–8 N range. On the nominal mission the tracking error converges to $2 \times 10^{-9}$ m and the motor forces stay between 1.72 and 2.65 N. With gains taken from the literature, the same mission would need between $-36$ and 46 N per motor. A Monte Carlo campaign over 500 random initial positions and attitudes (up to 175°) converges in 95.6 % of the cases. Every failure there comes from hitting the ground, which the analysed model does not contain. Stability claims are checked by a computer-algebra system instead of being asserted by hand, and every number in this report is regenerated by one command.

| **Key result** | **Value** |
|---|---|
| Steady-state tracking error | $2 \times 10^{-9}$ m (settles below 5 cm in 8.7 s) |
| Motor forces, nominal mission | 1.72–2.65 N, inside the 0–8 N motor range |
| Same mission, gains from the literature | $-36$ to 46 N, not realisable |
| Monte Carlo (500 runs, attitude errors up to 175°) | 95.6 % converge; 97.8 % never leave the motor range |
| Inside the certified attitude region (138 runs) | 100 % converge, as the analysis predicts |
| Symbolic identities verified by SymPy | 21 |
| Gain synthesis (6 per-axis LMI designs, open-source solver) | 0.7 s |
| Full reproduction of every result | one command, 111 s on a laptop |

## 1 Problem and requirements

A quadrotor of mass 0.8 kg must take off from rest at $(0, 0, 0.5)$ m and track the horizontal circle

$$p_d(t) = [\,3\cos(0.5\,t),\ 3\sin(0.5\,t),\ 3\,]^\top \text{ m},$$

heading along the tangent. Each of its four motors delivers a thrust between 0 and 8 N (hover needs 1.96 N per motor). The physics is integrated by MuJoCo [7] at 500 Hz, and the controller runs at the same rate.

| **ID** | **Requirement** | **How it is met** |
|---|---|---|
| R1 | Zero steady-state tracking error | Exact feedforward of the reference acceleration and of the desired-frame motion (Sec. 3) |
| R2 | Motor forces inside 0–8 N on the nominal mission | Gains synthesised from the motor budget by LMIs (Sec. 4) |
| R3 | Every stability claim backed by checkable evidence | Proofs verified by SymPy, certificates re-checked by tests (Sec. 5) |
| R4 | Robustness to the initial condition quantified | Monte Carlo over position and attitude errors (Sec. 6) |
| R5 | Reproducible by a third party with one command | `uv run python main.py` (Sec. 9) |

## 2 Architecture

The controller is a two-level cascade (Fig. 1). The outer loop turns the position error into a desired thrust vector. Its direction, together with the desired heading, defines the desired attitude $R_d$. The inner loop computes the torques that rotate the body onto $R_d$. A fixed allocation matrix then converts total thrust and torques into the four motor forces.

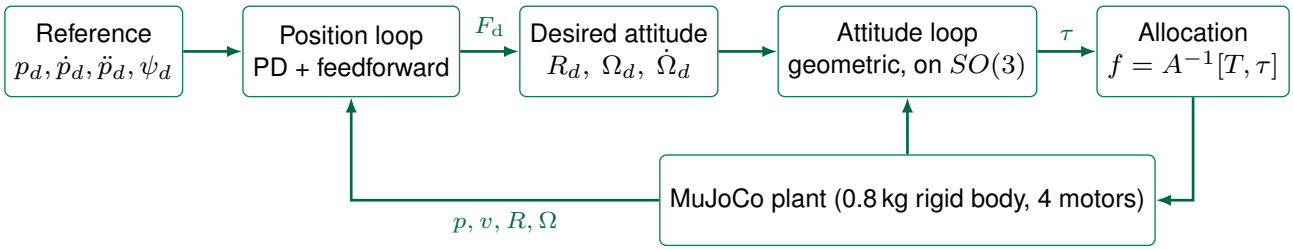



Figure 1: Control architecture. $F_\mathrm{d}$: desired thrust vector; its magnitude $T$ goes directly to the allocation.

**Software.** The guiding rule was to write only the parts that are specific to this problem and to rely on maintained libraries for everything else (Table 1). Vehicle parameters have a single source of truth: mass, inertia, motor positions, rotor torque coefficient and motor range are read from the MuJoCo model file. The allocation matrix in particular is MuJoCo's own actuator transmission, not a hand-written matrix.

| Need | Library | Role |
|---|---|---|
| Rigid-body physics, allocation | MuJoCo | Simulation; mass, inertia and allocation read from the model |
| Symbolic verification | SymPy | Derives and checks every identity of the stability analysis |
| Gain synthesis | cvxpy + Clarabel [3] | Semidefinite programs, open-source solver, no licence |
| Rotations, ODE integration | SciPy | Attitude sampling, unicycle integration (RK45) |
| Parallel Monte Carlo | joblib | One rollout per core |
| Data, plots, tests | pandas, matplotlib, pytest | Logs, figures, 29 automated tests |
| Environment | uv | Locked, reproducible dependency set |

Table 1: Off-the-shelf components. The project-specific code (controller, gain design, analysis, experiments) is about 1 100 lines of Python.

## 3 Control design

The design follows the geometric tracking controller of Lee et al. [6], with the weighted attitude error of [5] so that every axis can get its own gain.

**Position loop.** With $e_p = p - p_d$ and $e_v = \dot{p} - \dot{p}_d$,

$$F_\mathrm{des} = m\big(-K_v K e_p - K_v e_v + \ddot{p}_d + g e_3\big), \qquad T = \|F_\mathrm{des}\|. \tag{1}$$

The feedforward term $\ddot{p}_d$ supplies the centripetal acceleration. Without it, the vehicle settles on a smaller, lagging circle, with a 17 cm error in the previous version of this project.

**Attitude loop.** $R_d$ has its third axis along $F_\mathrm{des}$ and its first axis as close as possible to the heading. With the attitude error $R_e = R_d^\top R$,

$$\tau = \underbrace{-e_R - K_\Omega e_\Omega}_{\text{feedback}} + \underbrace{\Omega \times J\Omega - J\big(\hat{\Omega} R_e^\top \Omega_d - R_e^\top \dot{\Omega}_d\big)}_{\text{cancellation}}, \qquad e_R = \tfrac{1}{2}\big(G R_e - R_e^\top G\big)^\vee. \tag{2}$$

The cancellation terms remove the gyroscopic coupling and the motion of the desired frame. Without them the residual error does not vanish (1.3 mm RMS on the nominal mission) and the stability proof does not apply. The desired angular velocity $\Omega_d$ and its derivative come from the sequence of desired attitudes by finite differences. On the steady circle this is exact; during transients it is accurate to first order in the 2 ms control period.

## 4 Actuator-aware gain synthesis

Gains published for this controller were designed for a 4.3 kg vehicle with an inertia about 20 times larger. On this vehicle they demand $-36$ to 46 N per motor (Fig. 2, left), far outside what the motors can deliver. Rather than re-tuning by trial and error, the gains are computed from the motor budget.

**Budget.** Each motor force is $f_i = T/4 + \delta_i$, where $\delta_i$ is set by the torques. Reserving 1.5 N of each motor's range for torques, and splitting it among roll, pitch and yaw, gives torque budgets of 0.234 N m (roll, pitch) and 0.0288 N m (yaw: the rotor drag torque is small, so yaw authority is about ten times weaker). The remaining range bounds the correction acceleration to 3.0 m/s$^2$ horizontally and 2.31 m/s$^2$ vertically. If every loop respects its budget, every motor stays inside 0–8 N.

**Synthesis.** For each axis of each loop (a double integrator once the cancellation terms act) we solve the input-bounded state-feedback problem [2]: find $W \succ 0$ and $X$ with $K = XW^{-1}$ such that

$$AW + WA^\top + BX + X^\top B^\top \preceq -2\alpha W, \qquad \begin{bmatrix} 1 & x_0^\top \\ x_0 & W \end{bmatrix} \succeq 0, \qquad \begin{bmatrix} \bar{u}^2 & X \\ X^\top & W \end{bmatrix} \succeq 0, \tag{3}$$

and maximise the decay rate $\alpha$ by bisection. Every trajectory of the linear model that starts in the ellipsoid $\{x^\top W^{-1} x \leq 1\}$ stays in it, decays at rate $\alpha$, and never asks for more than $\bar{u}$. The design points $x_0$ are the mission start plus a margin. The result is a position loop with rates of 0.53–0.78 s$^{-1}$ and an attitude loop at 9.1 s$^{-1}$ (roll, pitch) and 2.1 s$^{-1}$ (yaw). The inner loop is therefore faster than the outer loop, as the cascade needs. The nominal mission then stays between 1.72 and 2.65 N per motor (Fig. 2, right), at the cost of a slower transient: 8.7 s instead of 3.3 s to reach 5 cm.

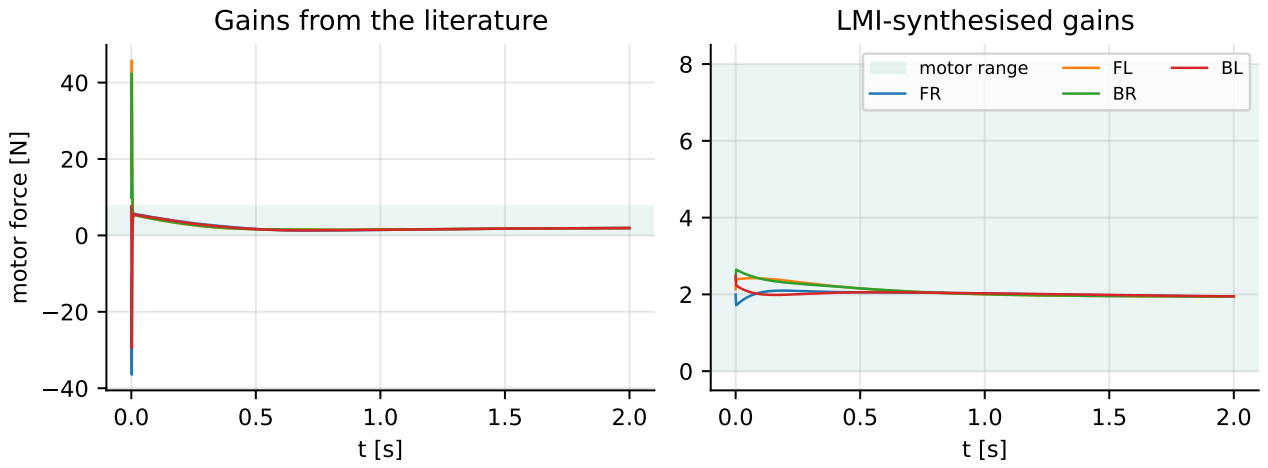



Figure 2: Motor forces during take-off. Left: gains from the literature. Right: LMI gains. Shaded: motor range.

## 5 Verification and validation

The project separates what is proved, what is checked numerically, what is observed in simulation and what is assumed (Table 2).

**What is not claimed.** The closed loop is not globally asymptotically stable. No continuous feedback on the rotation group can be [1]: a 180° attitude error is an equilibrium of any such controller. The analysis states its region explicitly instead.

**Symbolic verification.** The stability argument rests on a few exact identities: the attitude error function decreases at the rate $\dot{\Psi} = e_R \cdot e_\Omega$; the cancellation terms make the attitude error dynamics independent of the trajectory; and the only attitudes with $e_R = 0$ are the identity and 180° rotations. SymPy derives each identity from the equations of motion and the control law, and checks it for a generic matrix, so no small-angle approximation is involved. It also produces a concrete state at which the feedback-only law makes the Lyapunov function increase. That state shows why the cancellation terms are needed.

**Automated tests.** 29 `pytest` tests cover the symbolic identities, the LMI certificates (decay rate, design points inside the ellipsoid, input bound over the ellipsoid), and the closed loop. Among them: the mission converges, all

| Status | Statement | Evidence |
|---|---|---|
| Proved | The attitude error converges from every state with $V_2 < 0.092$, i.e. roll/pitch errors up to about 37° and any yaw error below 180° at zero initial rate | Lyapunov function and LaSalle; 21 identities verified by SymPy |
| Proved | The full closed loop is locally exponentially stable | Cascade structure and linearisation [4] |
| Proved | On the reduced linear models, the LMI gains respect the motor budget inside the certified ellipsoids | LMI certificates, re-checked by tests independently of the solver |
| Observed | Convergence and motor feasibility from large initial errors | MuJoCo simulation, Monte Carlo |
| Assumed | Motors can produce any force (no saturation in the analysis); $\Omega_d, \dot{\Omega}_d$ exact (implemented by finite differences) | Feasibility is reported when the range would be exceeded |

Table 2: Evidence behind each claim.

motor forces stay in range, the Lyapunov function never increases along the simulated trajectory, the finite-difference $\Omega_d$ matches its exact steady-state value, and the feedback-only law leaves a residual error.

## 6 Results

**Nominal mission.** The vehicle converges to the circle and the error decreases to machine precision (Fig. 3). The feedback-only law, without the cancellation terms, stalls at the millimetre level. The attitude Lyapunov function never increases along the simulated trajectory (Fig. 5), as the analysis predicts.

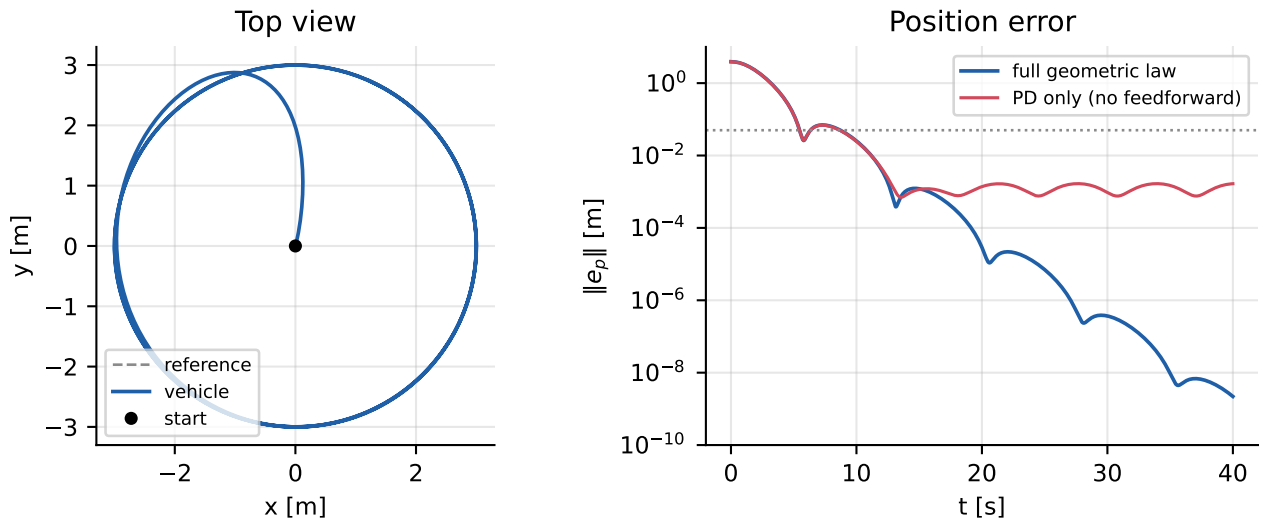



Figure 3: Nominal mission with the LMI gains. Left: top view. Right: position error, with and without the cancellation terms; dotted line at 5 cm.

**Robustness to the initial condition.** 500 runs start from random position offsets (up to 2 m) and random attitude errors (up to 175° about a random axis), at rest, and fly the same mission with the same gains (Fig. 4). 95.6 % converge. The other 4.4 % hit the ground during the recovery manoeuvre; the floor exists in the simulation but not in the analysed model. All 138 runs that start inside the certified attitude region converge, as the analysis predicts. 97.8 % of all runs never need more than the motors can deliver, even though the LMI certificate covered only the nominal take-off envelope.

## 7 Engineering review: defects found and fixed

A systematic review of an earlier version of the project found the defects below. Each one is now covered by a fix and, where possible, by an automated test.

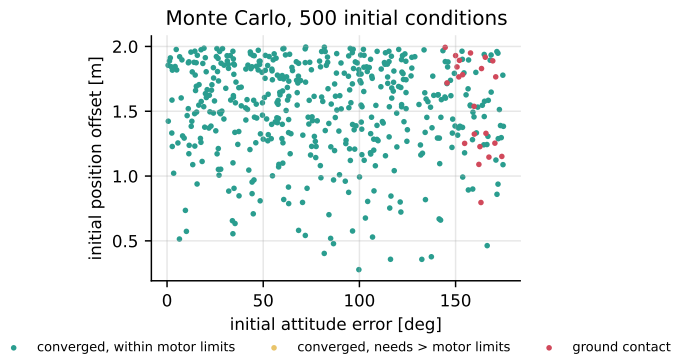



Figure 4: Monte Carlo: outcome for each initial condition.

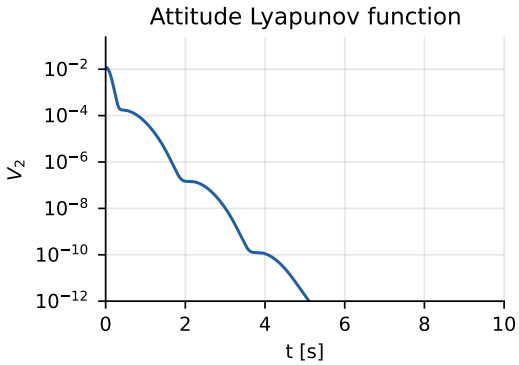



Figure 5: Attitude Lyapunov function on the nominal mission.

| Defect | Measured impact | Fix |
|---|---|---|
| Reference acceleration not fed forward | 17 cm steady-state error | Exact $\ddot{p}_d$ from the reference |
| Moment arm set to the arm length $L$ instead of $L/\sqrt{2}$ | Roll/pitch torques realised at 71 % | Allocation read from the MuJoCo model |
| Simulator silently clipped motor commands, while the analysis assumed no limits | Simulated and analysed systems differed | Limits stored in the model, enforced as a design constraint, reported in validation |
| Desired angular velocity approximated by the yaw rate | 3 cm residual error even with the acceleration fixed | $\Omega_d$, $\dot{\Omega}_d$ from $R_d$ |
| Gains copied from a 4.3 kg vehicle | Peaks above 150 N per motor | LMI synthesis from the motor budget |
| "Globally asymptotically stable" claimed with a hand-written derivative | Unsupported, and impossible on $SO(3)$ | Verified local results with explicit regions |
| About 900 lines of custom simulation, logging and plotting code | Maintenance cost | Replaced by MuJoCo, SciPy, joblib, pandas |

## 8 Limitations and next steps

- **Actuator saturation.** Motor limits are respected by design on the nominal mission, but are not enforced online. Next step: a control barrier function or a model predictive controller acting on the same allocation.
- **Unmodelled effects.** Motor dynamics, delays, aerodynamic drag, ground effect and sensor noise are not simulated. The state is assumed to be measured exactly. Next step: an estimator (EKF on IMU data) and a robustness study.
- **Path to hardware.** The controller is a pure function of state and reference at a fixed rate. It can be ported to a PX4 or ROS 2 software-in-the-loop setup before any flight test.

## 9 Reproducing the results

The only prerequisite is `uv`, which installs Python and the locked dependency set.

`cd mcav_project && uv run python main.py` — runs the symbolic checks, the LMI design, the simulations and the Monte Carlo, and writes the figures and `numbers.tex` used by this report (111 s).

`uv run pytest` — runs the automated tests.

`uv run jupyter lab` — opens `notebooks/final.ipynb`, a guided walkthrough of the same steps.

## References

[1] S. P. Bhat and D. S. Bernstein. A topological obstruction to continuous global stabilization of rotational motion and the unwinding phenomenon. Systems & Control Letters, 39(1):63–70, 2000.

[2] S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. Linear Matrix Inequalities in System and Control Theory. SIAM, 1994.

[3] S. Diamond and S. Boyd. CVXPY: A Python-embedded modeling language for convex optimization. Journal of Machine Learning Research, 17(83):1–5, 2016.

[4] H. K. Khalil. Nonlinear Systems. Prentice Hall, 3rd edition, 2002.

[5] T. Lee. Exponential stability of an attitude tracking control system on SO(3) for large-angle rotational maneuvers. Systems & Control Letters, 61(1):231–237, 2012.

[6] T. Lee, M. Leok, and N. H. McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In 49th IEEE Conference on Decision and Control, pages 5420–5425, 2010.

[7] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 5026–5033, 2012.